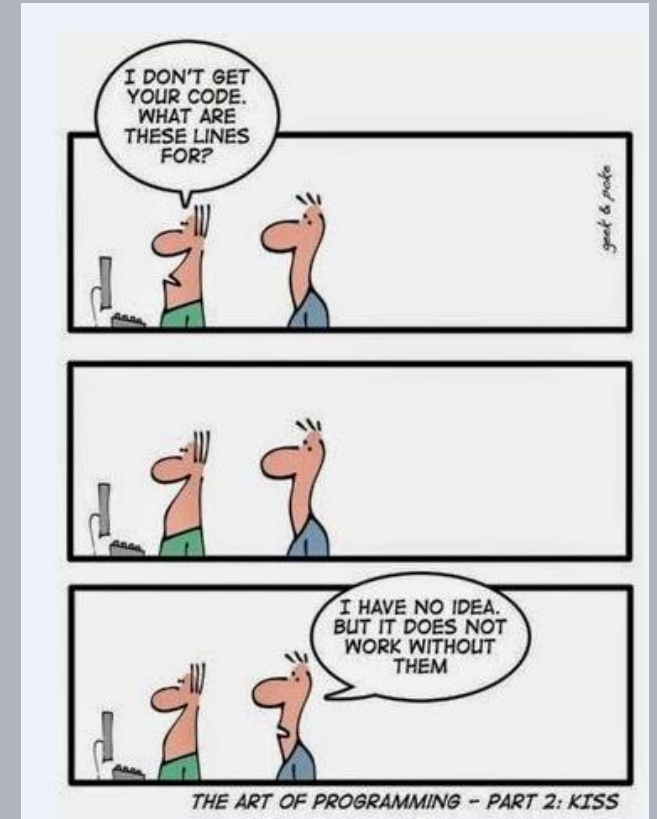


Advanced Software Design Techniques

Generic Programming and Templates

Prof. Dr.-Ing. Peter Fromm



Content

- The idea of Generalization
- Function Templates
- Class Templates
- Some Tricky Stuff



Specialization and Generalization

Programming Paradigm:



To improve reuse, code should be developed as **generic** as possible.

Specialization and Generalization

General rule of thumb:

- The more often a function is called by other functions, the more variant aspects must be considered.
- Especially: Library functions!
- The more often a function or class is reused for different projects, the more variants must be considered.

Specialization and Generalization

Mechanisms we have encountered so far:

- Preprocessor directives (#define)
- File variants
- Inheritance
- Overriding of functions
- Polymorphism
- Configuration files

CRingbuffer
-m_size:unsigned short -m_readIdx:unsigned short -m_writeIdx:unsigned short -m_fillLevel:unsigned short -m_pBuffer:int *
+CRingbuffer(size:unsigned short) +~CRingbuffer() +CRingbuffer(other:const CRingbuffer&) +put(data:int):bool +get(data:int &):bool

Discussion: What kind of flexibility can we achieve?
At which costs?

Generic Programming

Definition:

Generic programming is a style of computer programming in which algorithms are written in terms of **to-be-specified-later types** that are then instantiated when needed for specific types provided as parameters and was pioneered by Ada which appeared in 1983.

This approach permits writing common functions or types that differ only in the set of types on which they operate when used, thus **reducing duplication**. Software entities created using generic programming are known as generics in Ada, Eiffel, Java, C#, Visual Basic .NET and Haskell; templates in C++; and parameterized types in the influential 1994 book Design Patterns.

The authors of Design Patterns note that this technique, especially when combined with delegation, is very powerful but that "Dynamic, highly parameterized software is **harder to understand** than more static software." (Gang of Four 1995)

Templates – basic idea

Sometimes, we want to implement a function which behaves identical for different types and classes.

Consider for example:

- a function which returns the minimum value of 2 numbers

How would you solve this?



Traditional solution

Either implement a function for every set of parameters....

... or use type conversion!

... or use preprocessor macros!

- It's either very much work
- Or it's coming with loss of data
- Or requires very much memory (e.g. double instead of char)
- Or it's unsafe (macro)
- Or it's hard to debug



Better solution

As long as the **operations** of the function or class are **performed for all types in an identical manner**, you can use **template parameters**.

```
// A first template function
```

```
template <typename T>  
T min(T a, T b)  
{  
    cout << "Using template min: " ;  
    if (a < b) return a; else return b;  
}
```

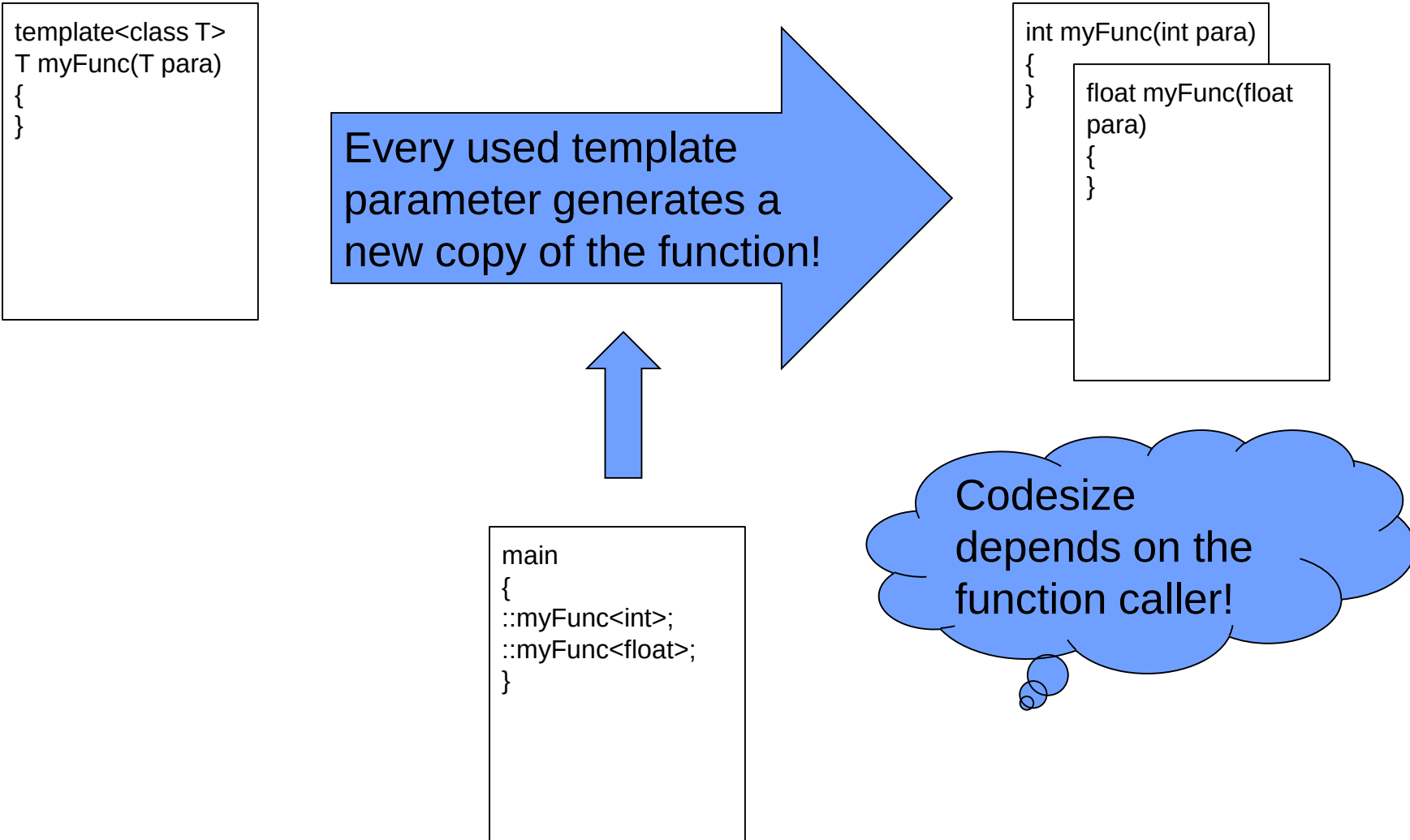
Observations

- Template function compiles for all types
- Similar behavior like macro
- “Wrong” result only for pointer operation

Technical background

- Template code is compiled twice
 - Template instantiation → generating instances as required
 - Template compilation → instantiated functions are compiled
- Templates are processed by the compiler, not by the preprocessor
 - Code is better checked during compilation
 - Compiler features like default parameters, function overloading etc. can be used
 - Code can easier be debugged

Technical Background



Inclusion Model

The implementation of template functions must be done in the header file!

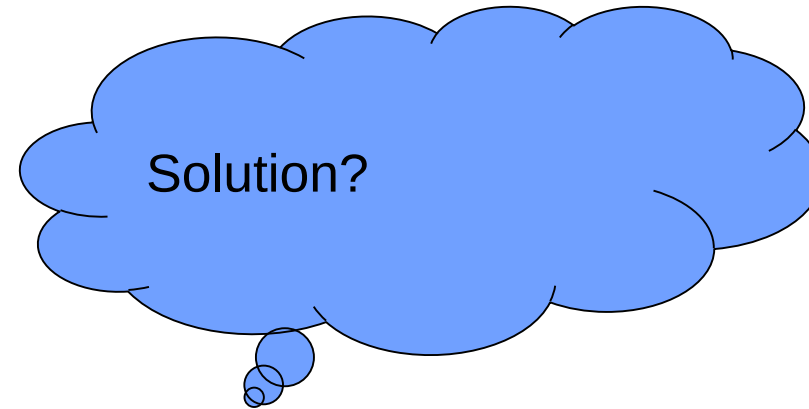
- Template code is “constructed” during the compilation similar like preprocessor functions and inline statements.
- I.e. the translation unit needs access to the source code for every object.
- As a consequence, they cannot be linked to the object like normal functions (i.e. implementation inside a cpp file). If you try it, you will receive a linker error.
- Problem: Code in Header files is not supported by the ANSI C++ standard

Beware of lazy instantiation!



Solving the “almost the same” problem

- The min function behaves the same for all provided types...
- ... but we would like to have a slightly different behavior for the pointers.



Template Function Specialization

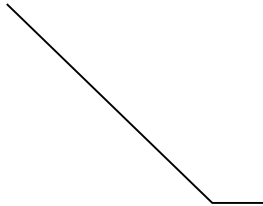
```
////////////////////////////////////  
// A first template function  
template <typename T>  
T min(T a, T b){  
    cout << "Using template min: " ;  
    if (a < b) return a; else return b;  
}  
  
////////////////////////////////////  
// And a specialisation for pointers  
template <typename T>  
T min(T* a, T* b) { ←  
    cout << "Using pointer specialisation of min: " ;  
    if (*a < *b) return *a; else return *b;  
}  
  
////////////////////////////////////  
// And a specialisation for string  
template < >  
string min(string a, string b) ←  
{  
    cout << "Using string specialisation of min: " ;  
    if (a.length() < b.length()) return a; else return b;  
}
```

We pass pointers but return dereferenced values

For strings, we want to compare the length

Sidenote on qualified names...

```
////////////////////////////////////  
// And a specialisation for 3 values  
template <typename T>  
T min(T a, T b, T c)  
{  
    cout << "Using template min with 3 values: " ;  
    return ::min(::min(a,b),c) ;  
}
```



Qualified name to avoid
call of global min
function.

Some Optimization Considerations

Until now, we passed all the data by value and did not protect it against modification. How about using

- call by reference
- const
- inline

```
////////////////////////////////////  
// An optimized template function  
template <typename T>  
inline T const& min(T const& a, T const& b)  
{  
    cout << "Using template min: " ;  
    if (a < b) return a; else return b;  
}
```

Argument Deduction

In the examples discussed so far, two parameters of the same type have been compared. But what happens, if we try to compare an int with a float variable?



Argument Deduction

Until now, the compiler created the correct instance of the function by checking the type of the provided parameter

```
uint32 u32a, u32b;  
...  
min(u32a, u32b)
```

is interpreted as

```
min<uint32>(u32a, u32b)
```



Argument Deduction

Problem of mixed parameters – the compiler does not know, how to instantiate the template function.

Solutions:

- Explicit instantiation: `min<float>(u32a, fb)`
- Parameter casting: `min((float)u32a, fb)`
- Two parameters: `inline T1 min(T1 const& a,
T2 const& b)`
- Solution 1 → good readability but implicit cast
- Solution 2 → explicit cast
- Solution 3 → many side effects due to implicit casting. Should be avoided!
- Combination of 1 and 2 is recommended

Argument Deduction

```
////////////////////////////////////
// A first template function
template <typename T>
inline T const& min(T const& a, T const& b)
{
    cout << "Using template min: " ;
    if (a < b) return a; else return b;
}

// Some data

    string s1("Peter");
    string s2("Paul");

// Some Testcases
    cout << " (with strings 1) " << ::min(s1,s2) << endl;
    cout << " (with strings 2) " << ::min("Paul","Mary") << endl;
```

//This works
//This does not work

Template Functions Conclusion

- Function templates define a family of functions for different template arguments
- Functions are instantiated during compile time depending on the function parameters (Automated deduction)
- Alternatively an explicit instance can be generated by using the template parameter
- Template functions can be overloaded

- KISS: Avoid more than 1 template parameter

- Optimization considerations
 - Call by value – robust, but class requires copy constructor
 - Call by reference – fast but dangerous
 - Const Call by reference – fast and secure, but all used methods must be declared const (legacy problem)
 - Inline – recommended for small functions

Template Classes

Similar to template functions, a template class defines a family of classes. In addition to the methods, attributes can be parameterized.



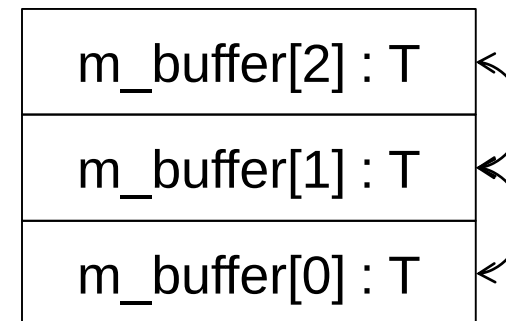
A Template Class CStorage Declaration

```
template <class T>
class CStorage {
private:
    int m_noElements;
    int m_max;
    T* m_buffer;

public:
    CStorage(int max = 5);
    ~CStorage();
    void sort();
    void print();
    bool add(T element);
};
```



Stores and sorts
any set of objects



Template Class CStorage

Method Implementation

```
template < class T >
CStorage<T>::CStorage(int max)
{
    m_noElements = 0;
    m_max = max;
    m_buffer = new T[m_max];
}

template < class T >
void CStorage<T>::sort()
{
    for (int i = 0; i < m_noElements; i++)
    {
        for (int l = i; l < m_noElements; l++)
        {
            if (m_buffer[l] < m_buffer[i])
            {
                //Exchange the data
                T temp = m_buffer[i];
                m_buffer[i]=m_buffer[l];
                m_buffer[l]=temp;
            }
        }
    }
}
```

CStorage = Name of class
CStorage<T> = Type of class

A template class

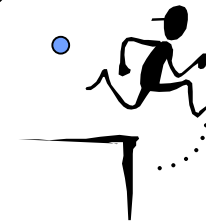
Usage and Contract

```
// A storage class storing integers  
CStorage<int> si(4);  
si.add(4);
```

Of course we can also
use own classes as
parameter types.

```
// A storage class storing coordinates  
CStorage<CCoordinate> sc(4);  
sc.add(CCoordinate(4,3));
```

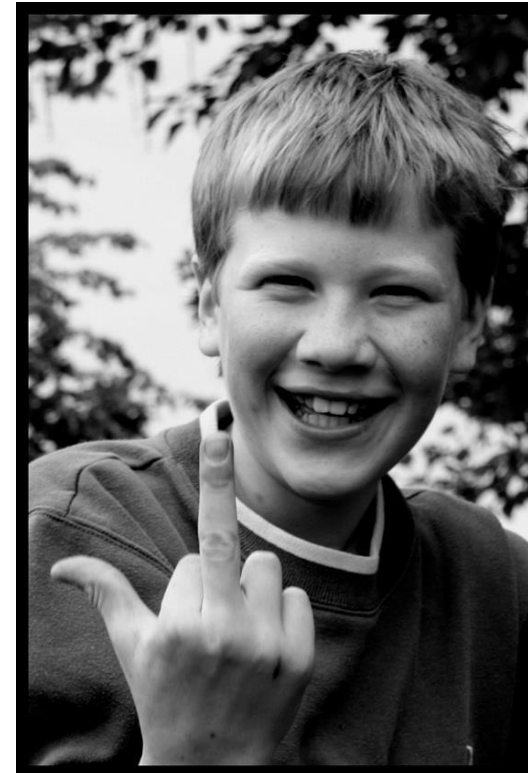
But we may need some
overloaded operators!



Conventions for type class

- Provide Constructor taking a “0” argument
- Provide overloaded operators (make it like an int)
 - << and >>
 - <, > and ==
 - Copy Constructor (or use default)
 - Assignment operator (or use default)

Make it like “int”



Nontype Template Parameter

In addition to type parameters, additional nontype class parameters may be added:

```
template <typename T, int MAXSIZE>
class CStorage
{
private:
    T m_array[MAXSIZE];
    //...
};
```

Advantages

- Array memory is inside the object
- No dynamic memory management required
- MPU support easy

Disadvantage

- Size cannot be modified during runtime
- Every new number generates a new instantiation of the template

Tricky Stuff



Default Template Arguments

Default template parameters can be used in a similar way like „normal“ default parameter

- from left to right, specifying the type or number (for nontype)
- upon calling parameters can be left blank from right to left

```
//Declaration
template <class T = int>
class CComplex
{
private:
    T m_real;
    T m_imaginary;
public:
    CComplex(T real = 0, T imaginary = 0);
    void print();
};

//Call
CComplex<> cu;
cu.print();
```

Template Cascade

Let's develop

- a class CComplex, which takes any variable type for the real and imaginary part of a complex number
- and a class CComplexArray, which stores a configurable number of these CComplex objects

CComplexArray<U,SIZE>

CComplex<T> →

T real	T imaginary
T real	T imaginary
T real	T imaginary

} SIZE



Template Template Parameter

Consider you want to create a class or function, which receives 2 type parameters:

- A storage type
- And a container type for the storage type

Straight forward implementation:

```
template <class T, class CONT>
void myFunc()
{
    CONT container;
...

```

Call: `myFunc<int, vector<int>>`

What happens in case of `myFunc<int, vector<float>>`

Template Template Parameter Better Solution - Principle

```
template <class T, template <class ELEM> class CONT >  
void myFunc()  
{
```

```
    CONT<T> container;  
    CONT<T>::const_iterator it;
```

Call: myFunc<int,**vector**>

Attention: This example describes only the basic concept, it does not compile, because a STL container requires two arguments.

Template Template Parameter Better Solution – Working Solution

```
template <class T, template <class ELEM, class ALLOC = allocator<ELEM> > class CONT >  
void myFunc()  
{
```

```
    CONT<T> container;  
    CONT<T>::const_iterator it;
```

Call: myFunc<int,vector>

This one will compile

Specialization of template methods

Sometimes, you need a specific method behavior depending on the type.

```
template <class T>
void CComplex<T>::print()
{
    cout << "Complex number: " << m_real
         << " + i" << m_imaginary << endl;
}
```

General

```
template <>
void CComplex<float>::print()
{
    cout << "Complex float number: " << m_real
         << " + i" << m_imaginary << endl;
}
```

Specialised for float

Specialization of template classes

Similar like template functions, template classes can be specialised, by providing a class with a specific type specifier. This is required if the attributes or API depends on the type.

```
template <>
class CComplex<string>
{
private:
    string m_complex;
    //...
};
```

Syntax is fragile. Use method implementation inside class body if it is not supported by the compiler.

```
//no template<> here, as the class is not a template anymore!
CComplex<string>::CComplex(string complexNumber)
{
    m_complex = complexNumber;
}
```

Partial Specialisation of template classes

```
template <class T1, class T2>
class CMyClass
{
public:
T1 m_v1;
T2 m_v2;
};
```

Generic class

```
template <class T>
class CMyClass<T,T>
{
public:
T m_v;
};
```

Variant for identical types

```
template <class T>
class CMyClass<T,int>
{
public:
T m_v1;
T m_v2;
int m_count;
};
```

Second parameter is int

Inclusion Model

Export Keyword

The C++ standard defines the keyword “export” for template code. The application is rather simple: just put the keyword export before the template declaration inside the header file and you can implement the class in the cpp file, just like non-template classes.

Problem: This is only supported by very few compilers and deprecated in the latest C++ standards.

export

```
template <class T>
class myclass
{...};
```

Explicit Instantiation

Sometimes it is necessary, to explicitly instantiate a certain class of a template – even if no object is existing for this class. Use case: testing purposes (compiler check and whitelisting of instantiations by placing the code in a cpp and add the explicit instantiation in the same file.

```
//Explicitly instantiate a template function  
template void print_typeof<double>(double const&)
```

```
//Explicitly instantiate MyClass for int  
template class MyClass<int>;
```

```
//Explicitly instantiate a constructor for int  
template class MyClass<int>::MyClass();
```

Good for testing (lazy instantiation) and if you want to put the code in the cpp-file. But you get a memory overhead!

Templates and static members

Remember:

- In nontemplate classes, all objects of a class share one instance of a static member, which is initialized at file scope.

Templates

- Each class instantiated from a class template has an own copy of each static class member!

Keyword “typename” versus “class”

template <class T> or template<typename T>

- Both possible

But consider the following code (used e.g. for STL):

```
template<typename T>
class MyClass {
    typename T::SubType* ptr;
...}
```

- This is a pointer to the subtype SubType of classtype T
- Without the typename, it would be a static variable of class T multiplied with ptr

Using “this” pointer

```
void print() { cout << "Hello external";}
```

```
template <class T>
class CBase
{
public:
    void print() { cout << "Hello base";}
};
```

```
template <class T>
class CDerived : public CBase<T>
{
public:
    void foo() {CBase<T>::print();}
    //void print() { cout << "Hello derived";}
};
```

Not all compilers will call the base method without explicit this-> or name qualifier

Any Type Initialization

```
class CMyClass
{
private:
    int m_x;
public:
    CMyClass(int x = 0) : m_x(x) {};
};

void foo()
{
    int i = 0;
    int j;
    CMyClass o1(4);
    CMyClass o2;
}
```

Initialization
values of
ANY type?



Zero Initialization

For a template function or class, the different initialization behavior for built-in types and classes must be considered.

- C++ provides a “constructor” for built in types
- `int x = int();`

```
template <typename T>
void foo()
{
    T i = T();
}
```

```
template <class T>
class CAnotherClass
{
private:
    T m_x;
public:
    CAnotherClass() : m_x()
    {} ;
};
```

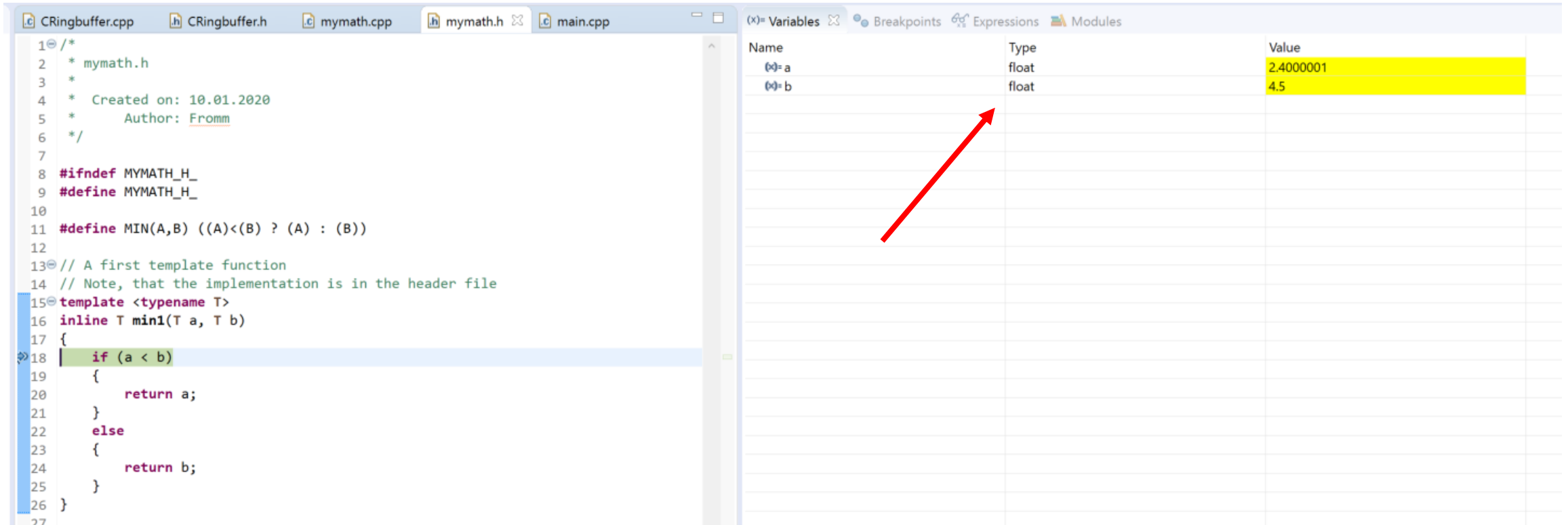
Debugging

Templates are great for effectively writing code. But what happens if it goes wrong?

- Cryptic debugger messages
- Even stranger runtime behaviour
- Diffusion of responsibility – is it the template class or the class passed as parameter?



Debugging



The screenshot shows a C++ IDE with a code editor on the left and a variables window on the right. The code editor displays the implementation of a `min1` function template in `mymath.h`. The function is defined as follows:

```
1  /*  
2   * mymath.h  
3   *  
4   * Created on: 10.01.2020  
5   * Author: Fromm  
6   */  
7  
8  #ifndef MYMATH_H_  
9  #define MYMATH_H_  
10  
11 #define MIN(A,B) ((A)<(B) ? (A) : (B))  
12  
13 // A first template function  
14 // Note, that the implementation is in the header file  
15 template <typename T>  
16 inline T min1(T a, T b)  
17 {  
18     if (a < b)  
19     {  
20         return a;  
21     }  
22     else  
23     {  
24         return b;  
25     }  
26 }  
27
```

The variables window on the right shows the current state of the program's variables:

Name	Type	Value
(x)= a	float	2.4000001
(x)= b	float	4.5

A red arrow points from the `if (a < b)` condition in the code editor to the variables window, indicating the state of the variables at that point in the execution.

Debugging

Most important rule:

- keep the structure as simple as possible
- develop (including testing) bottom up

Some extended techniques

- Policies and Traits



Golden Rules

- Keep the structure as simple as possible
- Document any restrictions / prerequisites of types being used with the template
- Consider and check memory consumption
- Use specialization as required, but try to minimize it
- Don't forget initialization

A template class represents a family of classes!

Questions?

